



Compiladores

Documento Técnico — Compilador para el lenguaje C1

Integrantes

López Asano Miguel Akira
Concha Zamora Marco Antonio
Hernández Rodríguez Oscar David
Celaya Nava Carlos Adrián
Cabrera Solís Irving Axel

Facultad de Ciencias, UNAM
Compiladores
16 de junio de 2026

Índice

1. Introducción	2
2. Descripción del Lenguaje	2
2.1. Tokens	2
2.2. Tipos de datos	2
2.3. Estructuras y sentencias soportadas	3
2.4. Expresiones y operadores	3
3. Gramática	3
3.1. Gramática Libre de Contexto	4
3.2. Conflictos	5
4. Análisis Semántico	5
4.1. Atributos	5
4.2. Reglas semánticas	6
4.3. Manejo de ámbitos	6
4.4. Manejo de tipos	7
5. Tablas	7
5.1. Tabla de símbolos	7
5.2. Tabla de tipos	8
5.3. Temporales y etiquetas	8
6. Código Intermedio (TAC)	8
7. Casos de Prueba	9
8. Ejecución	9
9. Limitaciones conocidas y trabajo pendiente	10

1 Introducción

Este documento describe el diseño e implementación del front-end de un compilador para el lenguaje **C1**, un subconjunto de C que incluye tipos básicos, arreglos multidimensionales, structs, funciones con parámetros y valor de retorno, y las estructuras de control usuales (if/else, while, for, break, return).

El compilador está implementado en C++ utilizando **Flex** para el análisis léxico y **Bison** para el análisis sintáctico y semántico dirigido por sintaxis. Como salida produce las tablas de símbolos y de tipos generadas durante la compilación, así como código intermedio de tres direcciones (TAC).

2 Descripción del Lenguaje

2.1 Tokens

La Tabla 1 resume las categorías de tokens reconocidas por el analizador léxico (lexer.ll).

Categoría	Tokens / ejemplos
Identificadores	ID ([a-zA-Z_] [a-zA-Z0-9_]*)
Literales	NUM, FLOAT_LITERAL, CHAR_LITERAL, TRUE, FALSE
Tipos	int, float, bool, char, struct
Control	def, if, else, while, for, break, return
Aritméticos	+ - * / % ¹
Relacionales	== != < > <= >=
Lógicos	&& !
Asignación	=
Delimitadores	; , . [] { } ()
Comentarios	// línea, /* bloque */

Cuadro 1: Tokens reconocidos por el analizador léxico

Ante un carácter no reconocido, el analizador léxico reporta:

Error lxico: caracter inesperado '\$' en linea 7

2.2 Tipos de datos

- **Básicos:** int, float, char, bool.

¹El operador % está reconocido por el lexer (MOD) pero aún no tiene una regla en la gramática que lo utilice.

- **Arreglos multidimensionales:** declarados como `T[n1][n2]...[nk] id;`. Cada dimensión se registra como un tipo array independiente en la tabla de tipos (Sección 5.2).
- **Structs:** declarados con `struct Nombre { ... } v1, v2, ...;`, generan un nuevo tipo con su propia tabla de símbolos para los campos.

2.3 Estructuras y sentencias soportadas

- Variables globales y locales, funciones con parámetros tipados y valor de retorno.
- Llamadas a función con verificación de número y tipo de argumentos.
- Acceso a campos de struct (`p.x`) e indexación de arreglos, incluyendo multidimensionales (`a[i][j][k]`).
- Asignación, `if/if-else`, `while`, `for` (estilo C), `break` y `return`.

2.4 Expresiones y operadores

La Tabla 2 muestra la precedencia y asociatividad de los operadores, de menor a mayor precedencia, tal como están declarados en `parser.yy`.

Precedencia	Operadores	Asociatividad
Más baja	=	Derecha
		Izquierda
	&&	Izquierda
	==, !=	Izquierda
	<, >, <=, >=	Izquierda
	+, -	Izquierda
	*, /	Izquierda
	! (negación lógica)	Derecha
	. (acceso a campo)	Izquierda
Más alta	[] (indexación)	Izquierda

Cuadro 2: Precedencia y asociatividad de operadores

3 Gramática

La gramática se implementa en `parser.yy` y está dirigida por sintaxis: cada regla tiene asociadas acciones semánticas que construyen las tablas de símbolos y tipos, y emiten código intermedio. A continuación se presenta la gramática libre de contexto completa (sin las acciones semánticas).

3.1 Gramática Libre de Contexto

$P \rightarrow D$

$D \rightarrow$
 FUNC D
 | $T L ; D$
 | **struct** $id \{ D \}$ STRUCT_VARS ; D
 | ϵ

$FUNC \rightarrow \text{def } T \text{ id } (F) \text{ BLOQUE}$

STRUCT_VARS $\rightarrow$ **id**
 | STRUCT_VARS , **id**
 | ϵ

$T \rightarrow B A$

$B \rightarrow$ **int** | **float** | **bool** | **char**

$A \rightarrow$ [**num**] A
 | ϵ

$L \rightarrow$ $L , \text{ id}$
 | **id** A

$F \rightarrow$ $F , T \text{ id}$
 | $T \text{ id}$
 | ϵ

$BLOQUE \rightarrow \{ S \}$

$S \rightarrow$ STMT S
 | ϵ

LVALUE $\rightarrow$ **id**
 | LVALUE [E]
 | LVALUE . **id**

STMT $\rightarrow$ LVALUE = $E ;$
 | **if** (E) STMT
 | **if** (E) STMT **else** STMT
 | **while** (E) STMT
 | **for** (**id** = $E ; E ; \text{ id} = E$) STMT
 | **break ;**
 | RETURN_STMT
 | BLOQUE
 | $T L ;$

$RETURN_STMT \rightarrow \text{return } E ;$

$E \rightarrow$ $E \text{ op } E$
 | **!** E
 | $E . \text{ id}$
 | (E)
 | **num** | **flit** | **clit** | **true** | **false**
 | LVALUE
 | **id** (ARGS)

donde $op \in \{+, -, *, /, <, <=, >, >=, ==, !=, \&\&, ||\}$ (ver Tabla 2)

$$\begin{array}{lcl} \text{ARGS} & \rightarrow & E \\ & | & E, \text{ARGS} \\ & | & \varepsilon \end{array}$$

Nota: existe además una producción muerta $T \rightarrow \text{register } \{ D \}$, declarada en el parser pero sin token `REGISTER` en el lexer, por lo que es inalcanzable. Se omite de la gramática anterior (ver Sección 9).

3.2 Conflictos

Al generar el parser con Bison se reporta un conflicto *shift/reduce*, correspondiente al clásico problema del **dangling else**: al analizar

```
if (E) if (E) STMT else STMT
```

no es claro a cuál `if` pertenece el `else`. La gramática resuelve este conflicto declarando las pseudo-precedencias `LOWER_THAN_ELSE` y `ELSE`, y marcando la regla $\text{STMT} \rightarrow \text{if } (E) \text{ STMT}$ (sin `else`) con `%prec LOWER_THAN_ELSE`. Esto hace que Bison prefiera *shift* sobre *reduce*, asociando el `else` con el `if` más cercano (comportamiento estándar de C).

4 Análisis Semántico

4.1 Atributos

Cada símbolo no terminal de la gramática tiene asociada una estructura `Attr` (`headers/Attributes.hpp`) con los siguientes campos relevantes:

- `tipo`: identificador del tipo (índice en la tabla de tipos).
- `base`: tipo base, usado para arreglos.
- `dir`: nombre/dirección o temporal asociado al valor.
- `ldir`: dirección del *lvalue* cuando difiere de `dir` (por ejemplo, en accesos indexados).
- `tam`: tamaño en memoria.
- `trueLabel`, `falseLabel`, `nextLabel`: etiquetas para control de flujo.
- `lista` / `listaDir`: listas de tipos y direcciones, usadas para parámetros y argumentos.
- `code`: lista de instrucciones TAC generadas por la subexpresión.

4.2 Reglas semánticas

Durante el recorrido dirigido por sintaxis se realizan, entre otras, las siguientes verificaciones:

- **Identificadores no declarados:** al usar un ID como *lvalue* o en una expresión, se consulta la pila de tablas de símbolos (`pilaTs.lookup`); si no existe, se reporta "Variable no declarada: <id>".
- **Redeclaración:** variables, funciones y structs duplicados en el mismo ámbito se reportan como "Función redeclarada: <id>" (y análogos para variables/structs).
- **Compatibilidad de tipos en asignación:** se valida con `tiposCompatibles`, que permite asignaciones `int -> float` pero no al revés ni entre tipos distintos sin relación.
- **Condición booleana:** las condiciones de `if` y `while` deben ser de tipo `bool` (`esBooleano`); en caso contrario se reporta "Condición no booleana en if/while".
- **Indexación válida:** al indexar un *lvalue* con `[ ]`, se verifica que su tipo tenga un tipo base (es decir, sea un arreglo); de lo contrario se reporta "Subíndice sobre tipo no arreglo".
- **Llamadas a función:** se verifica que el identificador exista y sea de categoría `func`, que el número de argumentos coincida con los parámetros declarados ("Cantidad incorrecta de argumentos en llamada a <id>"), y que cada argumento sea de tipo compatible con el parámetro correspondiente.
- **Tipo de retorno:** en `return E`; se valida que el tipo de `E` sea compatible con el tipo de retorno declarado de la función actual (`tipoReturn`).

Los errores semánticos se imprimen por `cerr` y, además, activan la bandera `global errorSem`, que se utiliza al final de la ejecución para determinar si el análisis terminó *exitosamente* o *con errores*.

4.3 Manejo de ámbitos

El manejo de ámbitos anidados se realiza mediante `PilaTs`, una pila de tablas de símbolos (`SymTab*`):

- `pilaTs.bottom()` es siempre la tabla **global**.
- Al entrar a una función o a un struct, se hace `push` de una nueva `SymTab` vacía, que se convierte en el ámbito activo (`top()`).
- Las búsquedas de identificadores (`lookup`, `lookupType`, `lookupDir`, etc.) recorren la pila desde el tope hacia el fondo, implementando la regla de *scoping* usual (variable local oculta a la global).

- Al salir de la función o struct, se hace `pop()` y la tabla resultante se imprime y/o se asocia al símbolo de la función o struct en la tabla global.

Junto con la pila de tablas se mantiene una pila de direcciones (`pilaDir`) para restaurar el contador de direcciones (`dir`) al salir de un ámbito, de forma que cada función reinicia su propio espacio de direcciones locales (offsets desde 0).

4.4 Manejo de tipos

La compatibilidad y promoción de tipos se centraliza en tres funciones auxiliares:

- `esNumerico(tipo)`: cierto si el tipo es `int` o `float`.
- `esBooleano(tipo)`: cierto si el tipo es `bool`.
- `tiposCompatibles(t1, t2)`: cierto si `t1 == t2`, o si `t1` es `float` y `t2` es `int` (asignación `int -> float`).
- `tipoDominante(t1, t2)`: regresa `float` si alguno de los dos operandos es `float`, e `int` en otro caso. Se usa para tipar el resultado de operaciones aritméticas mixtas.

5 Tablas

5.1 Tabla de símbolos

La tabla de símbolos (`SymTab`, en `headers/SymTab.hpp`) almacena, para cada identificador, una estructura `Sym` con:

- `dir`: dirección/offset asignado (o -1 para funciones).
- `tipo`: identificador del tipo en la tabla de tipos.
- `cat`: categoría ("`var`", "`param`", "`func`").
- `params`: lista de tipos de parámetros, sólo para funciones.

Cada función y cada struct tiene su propia `SymTab` (ver Sección de manejo de ámbitos), que se imprime de forma independiente bajo el encabezado `TABLA FUNCION <nombre>`.

5.2 Tabla de tipos

La tabla de tipos (TypeTab, en headers/TypeTab.hpp) asigna a cada tipo un identificador entero y una estructura Type con:

- name: nombre del tipo (int, float, bool, char, nombre del struct, o "array").
- tam: tamaño en bytes.
- numElems: número de elementos (para arreglos).
- tipoBase: tipo de los elementos (para arreglos).
- ts: puntero a la tabla de símbolos de los campos (para structs).

Los cuatro tipos básicos (int, float, bool, char) se registran al inicio con IDs 0-3. Cada struct y cada dimensión de un arreglo multidimensional generan una nueva entrada (tipo array) mediante addArrayType/addStructType.

5.3 Temporales y etiquetas

El generador de TAC (TAC.cpp) provee dos funciones:

- nuevaTemp(): genera nombres de variables temporales t0, t1, t2, ...
- nuevaLabel(): genera etiquetas L0, L1, L2, ... para saltos.

Las instrucciones generadas se acumulan en el vector global codigo y se imprimen al final de la ejecución bajo el encabezado ===== CODIGO INTERMEDIO =====.

6 Código Intermedio (TAC)

El código intermedio generado es de tres direcciones e incluye:

- Temporales (t0, t1, ...) para resultados intermedios de expresiones.
- Etiquetas (L0, L1, ...) y saltos (goto, ifFalse ... goto ...) para if, while y for.
- Llamadas a función: param <arg> por cada argumento, seguido de t = call <id>, <n>.
- return <expr> para sentencias de retorno.

Por ejemplo, para la expresión $a = b + c * d$; se genera:

```
t0 = c * d
t1 = b + t0
a = t1
```

7 Casos de Prueba

El repositorio incluye, en `src/test/`, los siguientes casos de prueba (Tabla 3), cumpliendo el mínimo solicitado de 5 programas válidos y 3 programas con cada tipo de error.

Cuadro 3: Casos de prueba incluidos en el repositorio

Categoría	Archivo	Resultado
Válidos	<code>validos/01_factorial.txt</code>	Exitoso
	<code>validos/02_fibonacci.txt</code>	Exitoso
	<code>validos/03_busqueda_lineal.txt</code>	Exitoso
	<code>validos/04_struct_punto.txt</code>	Exitoso
	<code>validos/05_bubble_sort.txt</code>	Exitoso
Errores léxicos	<code>err_lexico/01_arroba.txt</code>	Con errores
	<code>err_lexico/02_hash.txt</code>	Con errores
	<code>err_lexico/03_dolar.txt</code>	Con errores
Errores sintácticos	<code>err_sintactico/01_falta_semicolon.txt</code>	Con errores
	<code>err_sintactico/02_llave_sin_cerrar.txt</code>	Con errores
	<code>err_sintactico/03_condicion_incompleta.txt</code>	Con errores
Errores semánticos	<code>err_semantico/01_variable_no_declarada.txt</code>	Con errores
	<code>err_semantico/02_tipos_incompatibles.txt</code>	Con errores
	<code>err_semantico/03_variable_duplicada.txt</code>	Con errores

8 Ejecución

El compilador se construye desde la carpeta `src/` con:

```
make
```

lo cual genera el ejecutable `comp`. Para analizar un archivo fuente:

```
./comp archivo.txt
```

La salida incluye, en orden: las tablas de símbolos por función, la tabla de tipos, la tabla de símbolos global, los errores detectados (si los hay), el código intermedio generado, y un mensaje final indicando si el análisis terminó *exitosamente* o *con errores*.

9 Limitaciones conocidas y trabajo pendiente

- No se han implementado las funciones de E/S `puts(cadena)` y `putw(expresion)`
- No hay constantes de cadena (`"texto"`) ni otros tipos primitivos que existan en C.
- No está implementada la palabra reservada `continue`
- No es posible hacer casting de manera implícita
- No hay apuntadores